

Centro Federal de Educação Tecnológica de Minas Gerais

Engenharia de Computação

Computação Gráfica

Relatório de Desenvolvimento

Remoção de Fundo por Chroma Key

João Paulo da Cunha Faria

Maria Eduarda Teixeira Souza

Divinópolis - MG

Setembro de 2026

1 Introdução

O Chroma Key é uma técnica clássica de composição de imagens, amplamente utilizada em cinema, televisão e transmissões ao vivo, que consiste em capturar um objeto ou pessoa diante de um fundo de cor sólida e, posteriormente, substituir esse fundo por outra imagem ou vídeo. Do ponto de vista da Computação Gráfica, o desafio não está na captura da cena, mas sim na correta segmentação matemática entre o que é "objeto"e o que é "fundo a ser removido".

Embora a cor verde ou azul do fundo pareça, à primeira vista, um valor constante e fácil de identificar, a iluminação do ambiente introduz uma grande variação de tons sobre a mesma superfície: regiões mais próximas da fonte de luz aparecem como um verde claro (quase amarelado), enquanto os cantos e dobras do tecido produzem sombras em tons de verde escuro. Um algoritmo ingênuo que compare cor a cor no espaço RGB falha nessa tarefa, pois tons de verde claro e verde escuro ocupam posições distantes dentro do cubo RGB, apesar de pertencerem à mesma "família"de cor sob a percepção humana.

Este relatório documenta a implementação de um algoritmo de Chroma Key que resolve esse problema por meio da mudança de sistema de coordenadas de cor do cubo RGB para o cilindro HSV (Matiz, Saturação e Valor), permitindo isolar o fundo verde unicamente pelo seu ângulo de matiz, independentemente da variação de brilho causada pela iluminação da cena.

2 Modelagem Matemática e Fundamentos de Cor

Como discutido em aula, o espaço RGB não é adequado para segmentar o fundo verde, pois a iluminação do estúdio faz a mesma superfície variar de um verde claro a um verde musgo escuro, valores distantes entre si no cubo RGB. Já no cilindro HSV, essas variações afetam principalmente o canal de Valor (V), enquanto o Matiz (H) — a coordenada angular que identifica a cor em si — permanece praticamente constante. É essa propriedade que o algoritmo explora para isolar o fundo de forma robusta.

2.1 Critério de Detecção do Fundo Verde

O matiz do verde puro localiza-se em 120°. Definindo uma janela de tolerância ao redor desse valor (no código, de 90° a 150°) e normalizando-a para o domínio [0.0, 1.0] exigido pela função `colorsys.rgb_to_hsv` ($H_{norm} = H_{graus}/360$), obtém-se o intervalo de detecção do fundo.

Apenas o matiz, porém, não basta: pixels muito próximos do branco, do cinza ou do preto têm matiz indefinido e poderiam ser removidos por engano. Por isso, o algoritmo também exige uma saturação e um valor (brilho) mínimos. Assim, um pixel (H, S, V) é classificado como fundo removível quando:

$$(H_{min} \leq H \leq H_{max}) \ \wedge \ (S \geq S_{min}) \ \wedge \ (V \geq V_{min}) \tag{1}$$

2.2 Substituição do Pixel (Composição Alfa)

Uma vez identificado um pixel de fundo pela Equação 1, ele é substituído seguindo a lógica de composição de imagens com canal alfa ($C_{out} = \alpha \cdot C_F + (1 - \alpha) \cdot C_B$), já vista em aula. No algoritmo implementado essa decisão é binária pixel a pixel: os pixels de fundo recebem $\alpha = 0$ (tornando-se transparentes, ou sendo trocados diretamente pelo pixel correspondente da nova imagem de fundo, quando esta é fornecida) e os demais pixels, pertencentes ao objeto, mantêm $\alpha = 1$ (permanecem inalterados).

3 Metodologia

3.1 Materiais e Configuração

O projeto foi desenvolvido em *Python*, utilizando a biblioteca nativa `colorsys` (conversão entre espaços de cor) e a biblioteca `Pillow` (`PIL`) — especificamente os módulos `Image` e `ImageDraw` —, responsáveis

pela leitura e manipulação matricial dos pixels, operações formais de desenho e álgebra do canal alfa (RGBA). A instalação da dependência externa é feita via:

```
pip install Pillow
```

3.2 Estrutura de Diretórios

Para organizar o fluxo de entrada e saída de imagens, o script define três diretórios de trabalho, criados automaticamente caso não existam:

Listing 1: Definição e criação automática dos diretórios do projeto

```
DIR_FRENTE = os.path.join(DIRETORIO_SCRIPT, "imagens_com_tela_verde")
DIR_FUNDO = os.path.join(DIRETORIO_SCRIPT, "imagens_de_fundo")
DIR_SAIDA = os.path.join(DIRETORIO_SCRIPT, "imagens_processadas")

def garantir_pastas_existem():
    """Cria os diretorios necessarios caso nao existam."""
    os.makedirs(DIR_FRENTE, exist_ok=True)
    os.makedirs(DIR_FUNDO, exist_ok=True)
    os.makedirs(DIR_SAIDA, exist_ok=True)
```

3.3 Procedimentos e Scripts Utilizados

A função `aplicar_chroma_key` (Listagem 2) é o núcleo matemático do algoritmo. Ela converte a imagem de entrada para o modo `RGBA` e instancia um objeto de desenho através de `ImageDraw.Draw`, cumprindo as diretrizes da biblioteca para edição de imagens. A leitura das cores originais é feita via `load()` por questões de desempenho, enquanto a reescrita é delegada à ferramenta de desenho.

O algoritmo percorre a imagem pixel a pixel utilizando `colorsys.rgb_to_hsv` para extrair o matiz, a saturação e o valor, aplicando o critério da Equação 1. Quando uma imagem de fundo é fornecida, ela é redimensionada para as dimensões exatas da imagem principal. Se um pixel atende ao critério de "fundo verde", o método `draw.point()` é invocado para desenhar o pixel da imagem de fundo naquela mesma coordenada; caso contrário, desenha-se o pixel original com canal alfa igual a zero (transparente).

Listing 2: Função principal de aplicação do Chroma Key via espaço HSV utilizando ImageDraw

```
def aplicar_chroma_key(imagem_frente, imagem_fundo=None, h_min_graus=90,
                       h_max_graus=150, s_minima_pct=20, v_minima_pct=15):
    """Remove o fundo verde e substitui por uma nova imagem (se fornecida)
    ou deixa transparente."""
    imagem_rgba = imagem_frente.convert("RGBA")

    # Objeto draw utilizado para a escrita formal dos pixels
    draw = ImageDraw.Draw(imagem_rgba)

    # Acesso matricial apenas para leitura eficiente
    pixels_frente = imagem_rgba.load()
    largura, altura = imagem_rgba.size

    pixels_fundo = None
    if imagem_fundo:
        fundo_redimensionado = imagem_fundo.convert("RGBA").resize((largura,
            altura))
        pixels_fundo = fundo_redimensionado.load()

    h_min_norm = h_min_graus / 360.0
    h_max_norm = h_max_graus / 360.0
```

```
    s_minima_norm = s_minima_pct / 100.0
    v_minima_norm = v_minima_pct / 100.0

    for x in range(largura):
        for y in range(altura):
            r, g, b, a = pixels_frente[x, y]
            h, s, v = colorsys.rgb_to_hsv(r / 255.0, g / 255.0, b / 255.0)

            matiz_eh_verde = h_min_norm <= h <= h_max_norm
            cor_eh_relevante = s >= s_minima_norm and v >= v_minima_norm

            if matiz_eh_verde and cor_eh_relevante:
                if pixels_fundo:
                    # Usa o ImageDraw para desenhar o pixel do fundo
                    draw.point((x, y), fill=pixels_fundo[x, y])
                else:
                    # Usa o ImageDraw para desenhar um pixel transparente
                    draw.point((x, y), fill=(r, g, b, 0))

    return imagem_rgba
```

A orquestração principal (Listagem 3) percorre todas as imagens da pasta de entrada, permite ao usuário escolher interativamente uma imagem de fundo (ou aplicar apenas transparência) e delega o processamento em lote para a função `processar_imagens`, que salva cada resultado com o sufixo `_processado.png`.

Listing 3: Rotina principal de orquestração do processamento em lote

```
def processar_imagens(arquivos_frente, img_fundo):
    """Roda o processo de chroma key para todas as imagens de entrada
    e salva o resultado."""
    for nome_arquivo in arquivos_frente:
        caminho_entrada = os.path.join(DIR_FRENTE, nome_arquivo)
        img_frente = Image.open(caminho_entrada)
        imagem_processada = aplicar_chroma_key(img_frente, img_fundo)

        nome_base, _ = os.path.splitext(nome_arquivo)
        nome_saida = f"{nome_base}_processado.png"
        caminho_saida = os.path.join(DIR_SAIDA, nome_saida)
        imagem_processada.save(caminho_saida)

def main():
    """Funcao principal que orquestra o fluxo do programa."""
    garantir_pastas_existem()
    arquivos_frente = listar_imagens(DIR_FRENTE)

    if not arquivos_frente:
        print(f"Nenhuma imagem encontrada na pasta: '{DIR_FRENTE}'")
        return

    img_fundo = selecionar_fundo()
    processar_imagens(arquivos_frente, img_fundo)

if __name__ == "__main__":
    main()
```

3.4 Melhorias Implementadas Além do Escopo Base

Além do requisito mínimo do desafio (uso das bibliotecas `colorsys`, `Image` e `ImageDraw` do `Pillow` para remover um fundo verde sólido), este trabalho implementou os seguintes aprimoramentos:

- **Substituição dinâmica de fundo:** a função `selecionar_fundo` lista automaticamente todas as imagens disponíveis na pasta `imagens_de_fundo` e permite ao usuário escolher qual delas será utilizada na composição, sem a necessidade de alterar o código-fonte;
- **Modo de transparência (RGBA):** caso o usuário não informe nenhuma imagem de fundo, o algoritmo preserva o canal alfa dos pixels removidos como 0, gerando um recorte com fundo transparente, reaproveitável em qualquer outra composição (por exemplo, sobreposição em apresentações ou vídeos);
- **Parâmetros de tolerância ajustáveis:** os limites de matiz (h_min_graus, h_max_graus), saturação mínima e valor mínimo são parâmetros da função, permitindo calibrar a sensibilidade do algoritmo para diferentes condições de iluminação e diferentes tonalidades de fundo (verde ou azul), sem alterar a lógica central;
- **Processamento em lote:** a função `processar_imagens` percorre automaticamente todas as imagens válidas de uma pasta de entrada, aplicando o mesmo critério de remoção a um conjunto arbitrário de arquivos, sem intervenção manual por imagem.

4 Resultados e Análises

Para validar o algoritmo, foi utilizada uma fotografia de um cão fotografado diante de um fundo verde de estúdio (Figura 1) e uma imagem de uma paisagem litorânea (Figura 2) como novo plano de fundo.



Figura 1: Imagem de entrada: objeto fotografado diante de fundo verde sólido de estúdio.



Figura 2: Imagem de fundo escolhida para a composição final.

Executando a função `aplicar_chroma_key` com os parâmetros padrão ($H \in [90°, 150°]$, $S_{min} = 20\%$, $V_{min} = 15\%$) e sem uma imagem de fundo, obtém-se o recorte do objeto com canal alfa transparente no lugar do fundo verde (Figura 3). Essa saída é útil como um ativo reutilizável em outras composições.



Figura 3: Resultado do algoritmo com $\alpha = 0$ no fundo: recorte do objeto com transparência (fundo em xadrez/branco apenas para visualização).

Em seguida, fornecendo a imagem da praia (Figura 2) como parâmetro `imagem_fundo`, o mesmo critério de detecção do matiz verde foi aplicado, porém desta vez substituindo cada pixel identificado como fundo pelo pixel correspondente da nova imagem.O resultado final é apresentado na Figura 4.



Figura 4: Resultado final da composição: o fundo verde original foi substituído pela imagem da paisagem litorânea.

Observa-se que o algoritmo foi capaz de remover corretamente tanto as regiões mais claras quanto as regiões mais escuras (sombreadas) do fundo verde, confirmando a hipótese de que o matiz (Hue) se mantém estável mesmo sob variação de iluminação, ao contrário do que ocorreria com uma comparação direta no espaço RGB. Nas bordas do objeto — especialmente ao redor das orelhas e do pescoço do animal — nota-se um leve resquício esverdeado (fenômeno conhecido como *color spill*), causado pela reflexão da luz verde do fundo sobre a superfície do objeto fotografado. A supressão completa desse efeito exigiria uma etapa adicional de correção de cor nas bordas, não implementada no escopo deste trabalho.

5 Conclusão

O experimento demonstrou, na prática, por que a técnica de Chroma Key profissional não é implementada por meio de comparações diretas de cor no espaço RGB, mas sim através de uma mudança de sistema de coordenadas para o espaço cilíndrico HSV. Ao isolar a informação de matiz (Hue) da informação de brilho (Value), o algoritmo tornou-se robusto a variações de iluminação sobre o fundo verde, algo que seria impossível com um simples teste de igualdade de cor. A aplicação da equação de composição alfa de Porter e Duff permitiu, ainda, tanto gerar recortes com transparência quanto compor a imagem final sobre um novo plano de fundo, replicando digitalmente o efeito utilizado em produções audiovisuais profissionais.

Referências

[1] RODRIGO, Raulivan. **Fundamentos de Cor: Computação Gráfica**. Notas de Aula (07 - Fundamentos de Cor.pdf). Departamento de Computação, Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG), 2026.

[2] PORTER, Thomas; DUFF, Tom. **Compositing Digital Images**. Computer Graphics, v. 18, n. 3, p. 253–259, 1984.

[3] PYTHON SOFTWARE FOUNDATION. **colorsys — Conversions between color systems**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/colorsys.html.

[4] PILLOW CONTRIBUTORS. **Pillow (PIL Fork) Documentation**. Disponível em: https://pillow.readthedocs.io/.

[5] FOLEY, James D. et al. **Computer Graphics: Principles and Practice**. 2. ed. Addison-Wesley, 1996. (Capítulo sobre modelos de cor HSV e HSL).